20261008 模拟赛

题目名称	布丁	子序列	传送	划分
源代码/输入/输出文件名	pudding	subsequence	teleport	partition
测试点时限	2.0 s	2.0 s	1.0 s	1.0 s
空间限制	512 MB	512 MB	512 MB	512 MB
分值	100	100	100	100
子任务个数	6	7	6	5
题目类型	传统	传统	传统	传统

注意事项

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. `main` 函数的返回值类型必须是 `int`，程序正常结束时的返回值必须是 0。
3. 选手提交程序**无需建立子文件夹**。
4. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
5. 程序可使用的栈空间内存限制与题目的内存限制一致。
6. 评测时采用的机器配置为：$13\text{th Gen Intel(R) Core(TM) i5} - 1335\text{U } 1.30\text{ GHz}$，内存 16 GB。上述时限以此配置为准。
7. 只提供 Linux 格式附加样例文件。
8. **编译选项：**`-O2 -std=c++14`。

布丁（pudding）

题目描述

Alice 和 Bob 在购买布丁。

如果他们一共购买了 n 块布丁，则他们会按照如下规则分配布丁：

- 将 n 块布丁分别编号为 1 至 n；
- 对于编号为 i 的布丁：
  - 如果 i 是**奇数**，且 i 能**整除** n，则 Alice 得到这块布丁；
  - 否则如果 i 是**偶数**，且 i 能**整除** n，则 Bob 得到这块布丁；
  - 否则，这块布丁会被赠送给他们的好朋友 Catherine（即 Alice 和 Bob 都**不会**得到这块布丁）。

分配完成后，记 Alice 得到的布丁块数为 a，Bob 得到的布丁块数为 b。如果 $|a-b| \leq 2$，则他们认为购买 n 块布丁是**公平**的。

为了决定要购买多少块布丁，Alice 和 Bob 会向你提出 t 次如下询问：

- 给定正整数 l, r（**保证** $\sum(r-l) \leq 5 \times 10^7$），你需要求出，在 $[l, r]$ 中有多少个正整数 m，使得购买 m 块布丁是**公平**的。

输入格式

从文件 `pudding.in` 中输入。

第一行包含一个正整数 t。

接下来 t 行，每行包含两个正整数 l 和 r。

输出格式

输出到文件 `pudding.out` 中。

共 t 行，其中第 i 行包含一个非负整数，表示第 i 次询问的答案。

输入输出样例

样例 #1

输入 #1

```
2
12 16
1 100
```

输出 #1

```
3
60
```

样例解释 #1

对于第一次询问：

- 若购买 12 块布丁，则 Alice 分得布丁的编号集合为 $\{1,3\}$，共分得 2 块，Bob 分得布丁的编号集合为 $\{2,4,6,12\}$，共分得 4 块，由于 $|2-4|=2\le 2$，购买 12 块布丁**是**公平的；
- 若购买 13 块布丁，则 Alice 分得布丁的编号集合为 $\{1,13\}$，共分得 2 块，Bob 分得布丁的编号集合为 $\varnothing$，共分得 0 块，由于 $|2-0|=2\le 2$，购买 13 块布丁**是**公平的；
- 若购买 14 块布丁，则 Alice 分得布丁的编号集合为 $\{1,7\}$，共分得 2 块，Bob 分得布丁的编号集合为 $\{2,14\}$，共分得 2 块，由于 $|2-2|=0\le 2$，购买 14 块布丁**是**公平的；
- 若购买 15 块布丁，则 Alice 分得布丁的编号集合为 $\{1,3,5,15\}$，共分得 4 块，Bob 分得布丁的编号集合为 $\varnothing$，共分得 0 块，由于 $|4-0|=4>2$，购买 15 块布丁**不是**公平的；
- 若购买 16 块布丁，则 Alice 分得布丁的编号集合为 $\{1\}$，共分得 1 块，Bob 分得布丁的编号集合为 $\{2,4,8,16\}$，共分得 4 块，由于 $|1-4|=3>2$，购买 16 块布丁**不是**公平的。

综上所述，在 $[12,16]$ 中有 3 个正整数 m，使得购买 m 块布丁是公平的。因此第一次询问的答案为 3。

样例 #2

见下发文件中的 `pudding/pudding2.in` 与 `pudding/pudding2.ans`。

该样例满足子任务 1 的限制。

样例 #3

见下发文件中的 `pudding/pudding3.in` 与 `pudding/pudding3.ans`。

该样例满足子任务 2 的限制。

样例 #4

见下发文件中的 `pudding/pudding4.in` 与 `pudding/pudding4.ans`。

该样例满足子任务 3 的限制。

样例 #5

见下发文件中的 `pudding/pudding5.in` 与 `pudding/pudding5.ans`。

该样例满足子任务 4 的限制。

样例 #6

见下发文件中的 `pudding/pudding6.in` 与 `pudding/pudding6.ans`。

该样例满足子任务 5 的限制。

样例 #7

见下发文件中的 `pudding/pudding7.in` 与 `pudding/pudding7.ans`。

该样例满足子任务 6 的限制。

数据范围

对于所有数据，保证 $1 \le t \le 5000$，$1 \le l \le r \le 10^9$，$\sum(r-l) \le 5 \times 10^7$。

子任务编号	分值	$r \le$	$\sum(r-l) \le$	子任务依赖
1	10	5000	3×10^6	无
2	10	10^5	3×10^6	1
3	30	3×10^6	3×10^6	$1, 2$
4	10	5×10^7	3×10^6	$1, 2, 3$
5	20	10^9	3×10^6	$1, 2, 3, 4$
6	20	10^9	5×10^7	$1, 2, 3, 4, 5$

子序列（subsequence）

题目描述

Alice 有一个非空正整数序列 $A = [a_1, a_2, a_3, \ldots, a_n]$。

她取出了序列 A 的全部 $2^n - 1$ 个非空**子序列**，将它们按**字典序**从小到大排序后，依次记为 $S_1, S_2, S_3, \ldots, S_{2^n-1}$。

给定正整数 k，她希望你求出 $S_1, S_2, S_3, \ldots, S_k$。

良心的 Alice 很快意识到你无法在时间限制内输出如此多的数。因此，给定正整数 b 和 m，对于长度为 l 的非空正整数序列 $C = [c_1, c_2, c_3, \ldots, c_l]$，Alice 按如下方式定义了 C 的哈希值：

$$h(C) = (\sum_{i=1}^{l}(c_i \times b^{l-i})) \bmod m$$

你只需要依次求出 $h(S_1), h(S_2), h(S_3), \ldots, h(S_k)$ 即可。

输入格式

从文件 `subsequence.in` 中输入。

第一行包含四个正整数 n, k, b, m。

第二行包含 n 个正整数 $a_1, a_2, a_3, \ldots, a_n$。

输出格式

输出到文件 `subsequence.out` 中。

共 k 行，其中第 i 行包含一个非负整数 $h(S_i)$。

输入输出样例

样例 #1

输入 #1

```
3 4 2 3
1 3 1
```

输出 #1

```
1
1
0
2
```

样例解释 #1

对于该样例，我们有 $S_1 = [1]$，$S_2 = [1]$，$S_3 = [1, 1]$，$S_4 = [1, 3]$，$S_5 = [1, 3, 1]$，$S_6 = [3]$，$S_7 = [3, 1]$。

注意：虽然 S_1 和 S_2 的值相同，但它们选取的下标不同，因此被视作**不同**的子序列。

接下来我们需要计算 S_1, S_2, S_3 和 S_4 的哈希值：

- $h(S_1) = (1 \times 2^0) \bmod 3 = 1$，因此在第一行输出 1；
- $h(S_2) = (1 \times 2^0) \bmod 3 = 1$，因此在第二行输出 1；
- $h(S_3) = (1 \times 2^1 + 1 \times 2^0) \bmod 3 = 0$，因此在第三行输出 0；
- $h(S_4) = (1 \times 2^1 + 3 \times 2^0) \bmod 3 = 2$，因此在第四行输出 2。

样例 #2

见下发文件中的 `subsequence/subsequence2.in` 与 `subsequence/subsequence2.ans`。

该样例满足子任务 1 的限制。

样例 #3

见下发文件中的 `subsequence/subsequence3.in` 与 `subsequence/subsequence3.ans`。

该样例满足子任务 2 的限制。

样例 #4

见下发文件中的 `subsequence/subsequence4.in` 与 `subsequence/subsequence4.ans`。

该样例满足子任务 3 的限制。

样例 #5

见下发文件中的 `subsequence/subsequence5.in` 与 `subsequence/subsequence5.ans`。

该样例满足子任务 4 的限制。

样例 #6

见下发文件中的 `subsequence/subsequence6.in` 与 `subsequence/subsequence6.ans`。

该样例满足子任务 5 的限制。

样例 #7

见下发文件中的 `subsequence/subsequence7.in` 与 `subsequence/subsequence7.ans`。

该样例满足子任务 6 的限制。

样例 #8

见下发文件中的 `subsequence/subsequence8.in` 与 `subsequence/subsequence8.ans`。

该样例满足子任务 7 的限制。

数据范围

对于所有数据，保证 $1 \leq n, k, b, m, a_i \leq 10^6$，$k \leq 2^n - 1$。

子任务编号	分值	$n \leq$	$k \leq$	$a_i \leq$	特殊性质	子任务依赖
1	10	15	$2^{15} - 1$	30	无	无
2	10	2000	2000	10^6	A	无
3	10	2000	2000	30	无	无
4	20	10^5	10^5	30	无	$1, 3$
5	10	10^5	10^5	10^6	A	2
6	10	10^5	10^5	10^6	无	$1, 2, 3, 4, 5$
7	30	10^6	10^6	10^6	无	$1, 2, 3, 4, 5, 6$

特殊性质 A：保证 a_i 互不相同。

传送（teleport）

题目描述

Bob 在练习传送。

Bob 练习传送的场地可以被视作一棵 n 个节点的有根树，其中节点的编号为 0 至 $n-1$，且节点 0 为根节点。树上的每条边有一个非负整数边权。

如果 Bob 的**熟练度**为 x，那么 Bob 能够从节点 u **传送**至节点 v，当且仅当以下两个条件**均成立**：

- 节点 u 是节点 v 的祖先，或节点 v 是节点 u 的祖先；
- 从节点 u 到节点 v 的简单路径上的所有边的边权**异或和小于等于** x。

Bob 向你提出了 q 次如下询问：

- 给定 u 和 v，你需要求出，为了使得 Bob 能够从节点 u 经过**零次、一次或多次传送**到达节点 v，他的**熟练度**至少需要是多少。特别地，当 Bob 能够从节点 u 经过**零次传送**到达节点 v 时，我们认为**答案为** 0。

输入格式

从文件 `teleport.in` 中输入。

第一行包含一个正整数 n。

第二行包含 $n-1$ 个非负整数 $p_1, p_2, p_3, \ldots, p_{n-1}$，其中 p_i 表示节点 i 在树上的父节点的编号。

第三行包含 $n-1$ 个非负整数 $w_1, w_2, w_3, \ldots, w_{n-1}$，其中 w_i 表示树上连接节点 i 与 p_i 的边的边权。

第四行包含一个正整数 q。

接下来 q 行，每行包含两个正整数 u 和 v。

输出格式

输出到文件 `teleport.out` 中。

共 q 行，其中第 i 行包含一个非负整数，表示第 i 次询问的答案。

输入输出样例

样例 #1

输入 #1

```
6
0 1 0 1 2
3 2 0 2 1
4
2 4
3 0
1 1
0 5
```

输出 #1

```
1
0
0
0
```

样例解释 #1

对于第一次询问，以下传送方式可以使得 Bob 在熟练度为 1 的情况下从节点 2 到达节点 4：

- 从节点 2 传送到节点 0，其中节点 0 是节点 2 的祖先，且从节点 2 到节点 0 的简单路径上的所有边的边权异或和为 $2 \oplus 3 = 1 \leq 1$；
- 从节点 0 传送到节点 4，其中节点 0 是节点 4 的祖先，且从节点 0 到节点 4 的简单路径上的所有边的边权异或和为 $3 \oplus 2 = 1 \leq 1$。

注意：由于节点 2 不是节点 4 的祖先，且节点 4 不是节点 2 的祖先，Bob 无法直接从节点 2 传送到节点 4。

对于第二次询问，以下传送方式可以使得 Bob 在熟练度为 0 的情况下从节点 3 到达节点 0：

- 从节点 3 传送到节点 0，其中节点 0 是节点 3 的祖先，且从节点 3 到节点 0 的简单路径上的所有边的边权异或和为 $0 \leq 0$。

对于第三次询问，由于 Bob 能够从节点 1 经过零次传送到达节点 1，答案应为 0。

对于第四次询问，以下传送方式可以使得 Bob 在熟练度为 0 的情况下从节点 0 到达节点 5：

- 从节点 0 传送到节点 5，其中节点 0 是节点 5 的祖先，且从节点 0 到节点 5 的简单路径上的所有边的边权异或和为 $3 \oplus 2 \oplus 1 = 0 \leq 0$。

样例 #2

见下发文件中的 `teleport/teleport2.in` 与 `teleport/teleport2.ans`。

该样例满足子任务 1 的限制。

样例 #3

见下发文件中的 `teleport/teleport3.in` 与 `teleport/teleport3.ans`。

该样例满足子任务 2 的限制。

样例 #4

见下发文件中的 `teleport/teleport4.in` 与 `teleport/teleport4.ans`。

该样例满足子任务 3 的限制。

样例 #5

见下发文件中的 `teleport/teleport5.in` 与 `teleport/teleport5.ans`。

该样例满足子任务 4 的限制。

样例 #6

见下发文件中的 `teleport/teleport6.in` 与 `teleport/teleport6.ans`。

该样例满足子任务 5 的限制。

样例 #7

见下发文件中的 `teleport/teleport7.in` 与 `teleport/teleport7.ans`。

该样例满足子任务 6 的限制。

数据范围

对于所有数据，保证 $2 \le n \le 5 \times 10^4$，$1 \le q \le 10^5$，$0 \le p_i \le i-1$，$0 \le w_i \le 2^{20}-1$，$0 \le u, v \le n-1$。

子任务编号	分值	$n \le$	$w_i \le$	子任务依赖
1	10	10	$2^{20}-1$	无
2	10	200	$2^{20}-1$	1
3	20	10^4	$2^{20}-1$	$1,2$
4	10	5×10^4	1	无
5	20	5×10^4	127	4
6	30	5×10^4	$2^{20}-1$	$1,2,3,4,5$

划分（partition）

题目描述

Catherine 喜欢连续和上升。她称一个长度为 m 的非空正整数序列 $A = [a_1, a_2, a_3, \ldots, a_m]$ 是**好**的，当且仅当对于所有 $2 \leq i \leq m$，都有 $a_i = a_{i-1} + 1$。特别地，长度为 1 的正整数序列总是**好**的。

Catherine 还喜欢划分。给定正整数 n，对于一个 1 至 n 的**排列** Q，她定义 $f(Q)$ 为如下问题的答案：

- 至少需要将 Q 划分为多少个**非空子段**，才能使得每个划分出的子段都是**好**的?

Catherine 给了你一个 1 至 n 的**排列** $P = [p_1, p_2, p_3, \ldots, p_n]$。对于 $[1, n]$ 中的每个正整数 k，你需要求出，有多少个 1 至 n 的**排列** P' **同时满足**以下两个条件：

- P' 的**字典序严格小于** P；
- $f(P') = k$。

答案需要对给定的正整数 m 取模，m **不一定是质数**。

输入格式

从文件 `partition.in` **中输入。**

第一行包含两个正整数 n 和 m。

第二行包含 n 个正整数 $p_1, p_2, p_3, \ldots, p_n$。

输出格式

输出到文件 `partition.out` **中。**

共一行，包含 n 个非负整数，其中第 i 个非负整数表示满足 P' 的字典序严格小于 P，且 $f(P') = i$ 的 1 至 n 的排列 P' 的数量对 m 取模的结果。

输入输出样例

样例 #1

输入 #1

```
3 233
3 2 1
```

输出 #1

```
1 2 2
```

样例解释 #1

满足 P' 的字典序严格小于 P，且 $f(P')=1$ 的 1 至 n 的排列 P' 只有 $[1,2,3]$。

满足 P' 的字典序严格小于 P，且 $f(P')=2$ 的 1 至 n 的排列 P' 有 $[2,3,1]$ 和 $[3,1,2]$。前者可被划分为 $[2,3]$ 和 $[1]$，后者可被划分为 $[3]$ 和 $[1,2]$。

满足 P' 的字典序严格小于 P，且 $f(P')=3$ 的 1 至 n 的排列 P' 有 $[1,3,2]$ 和 $[2,1,3]$。$[1,3,2]$ 不能被划分为 $[1,2]$ 和 $[3]$，因为 $[1,2]$ 不是 $[1,3,2]$ 的子段。

样例 #2

见下发文件中的 `partition/partition2.in` 与 `partition/partition2.ans`。

该样例满足子任务 1 的限制。

样例 #3

见下发文件中的 `partition/partition3.in` 与 `partition/partition3.ans`。

该样例满足子任务 2 的限制。

样例 #4

见下发文件中的 `partition/partition4.in` 与 `partition/partition4.ans`。

该样例满足子任务 3 的限制。

样例 #5

见下发文件中的 `partition/partition5.in` 与 `partition/partition5.ans`。

该样例满足子任务 4 的限制。

样例 #6

见下发文件中的 `partition/partition6.in` 与 `partition/partition6.ans`。

该样例满足子任务 5 的限制。

数据范围

对于所有数据，保证 $1\le n\le 2000$，$10\le m\le 10^9$，$P=[p_1,p_2,p_3,\ldots,p_n]$ 是 1 至 n 的排列。

子任务编号	分值	$n \leq$	特殊性质	子任务依赖
1	10	10	无	无
2	20	80	无	1
3	30	300	无	$1,2$
4	10	2000	A	无
5	30	2000	无	$1,2,3,4$

特殊性质 A：保证 $p_i = n - i + 1$。